Lab Assignment 12.5

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
  - Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
  - Ask AI to include time complexity and space complexity in the function docstring.
  - Verify the generated code with test cases.
- Expected Output:
  - A functional Python script implementing Merge Sort with proper documentation.

Code :

merge.py

week 12 > merge.py > merge_sort

```python
#write a python program to create a function merge_sort(arr) that sorts a list i
from heapq import merge
def merge_sort(arr):
    """
    Sorts a list in ascending order using the merge sort algorithm.

    Time Complexity: O(n log n) - where n is the number of elements in the list.
    Space Complexity: O(n) - due to the temporary arrays used during merging.

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: A new sorted list in ascending order.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
```

week 12 > merge.py > ...

```python
    return merge(left_half, right_half)
def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

week 12 > merge.py > ...

```python
    return result
# Test cases
if __name__ == "__main__":
    test_cases = [
        [38, 27, 43, 3, 9, 82, 10],
        [5, 2, 9, 1, 5, 6],
        [12, 11, 13, 5, 6, 7],
        [],
        [1],
        [2, 1]
    ]

    for i, test in enumerate(test_cases):
        sorted_list = merge_sort(test)
        print(f"Test case {i + 1}: {sorted_list}")
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/User
e.py"
Test case 1: [3, 9, 10, 27, 38, 43, 82]
Test case 2: [1, 2, 5, 5, 6, 9]
Test case 3: [5, 6, 7, 11, 12, 13]
Test case 4: []
Test case 5: [1]
Test case 6: [1, 2]
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
  - Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.
  - Include docstrings explaining best, average, and worst-case complexities.
  - Test with various inputs.
- Expected Output:
  - Python code implementing binary search with AI-generated comments and docstrings.

Code :

week 12 > binary.py > ...

```python
#write a python program to create a function binary_search(arr, target) returning the index of the
def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the index of the target element.

    Time Complexity:
    - Best Case: O(1) - when the target is at the middle of the list.
    - Average Case: O(log n) - where n is the number of elements in the list.
    - Worst Case: O(log n) - when the target is not present in the list or is at one of the ends.

    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    arr (list): A sorted list in which to search for the target.
    target: The element to search for.

    Returns:
    int: The index of the target if found, otherwise -1.
    """
```

```python
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/A
ry.py"
Test case 1: Target 3 in [1, 2, 3, 4, 5] -> Index: 2
Test case 2: Target 1 in [1, 2, 3, 4, 5] -> Index: 0
Test case 3: Target 5 in [1, 2, 3, 4, 5] -> Index: 4
Test case 4: Target 6 in [1, 2, 3, 4, 5] -> Index: -1
Test case 5: Target 1 in [] -> Index: -1
Test case 6: Target 1 in [1] -> Index: 0
Test case 7: Target 0 in [1] -> Index: -1
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

Code :

week 12 > smart healthcare.py > ...

```python
def search_appointment(appointments, appointment_id):
    """
    Searches for an appointment by its ID.

    Time Complexity: O(n) - where n is the number of appointments.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    appointments (list): A list of appointment records (dictionaries).
    appointment_id (str): The ID of the appointment to search for.

    Returns:
    dict: The appointment record if found, otherwise None.
    """
    for appointment in appointments:
        if appointment['appointment_id'] == appointment_id:
            return appointment
    return None
```

week 12 > smart healthcare.py > ...

```python
def sort_appointments(appointments, key):
    """
    Sorts appointments based on a specified key (either 'time' or 'fee').

    Time Complexity: O(n log n) - where n is the number of appointments.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    appointments (list): A list of appointment records (dictionaries).
    key (str): The key to sort by ('time' or 'fee').

    Returns:
    list: The sorted list of appointment records.
    """
    return sorted(appointments, key=lambda x: x[key])
```

```python
# Test cases
if __name__ == "__main__":
    appointments = [
        {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 10:00', 'f
        {'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'f
        {'appointment_id': 'A003', 'patient_name': 'Alice Smith', 'doctor_name': 'Dr. Johnson', 'time': '2024-07-01 09:36
    ]

    # Test search_appointment
    print(search_appointment(appointments, 'A002'))  # Should return the appointment record for Jane Doe
    print(search_appointment(appointments, 'A004'))  # Should return None

    # Test sort_appointments by time
    sorted_by_time = sort_appointments(appointments, key='time')
    print("Appointments sorted by time:")
    for appointment in sorted_by_time:
        print(appointment)

    # Test sort_appointments by fee
    sorted_by_fee = sort_appointments(appointments, key='fee')
    print("Appointments sorted by fee:")
    for appointment in sorted_by_fee:
        print(appointment)
```

Output :

```
t healthcare.py"
{'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'fee': 150}
None
Appointments sorted by time:
{'appointment_id': 'A003', 'patient_name': 'Alice Smith', 'doctor_name': 'Dr. Johnson', 'time': '2024-07-01 09:30', 'fee': 120}
{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 10:00', 'fee': 100}
{'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'fee': 150}
Appointments sorted by fee:
{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 10:00', 'fee': 100}
{'appointment_id': 'A003', 'patient_name': 'Alice Smith', 'doctor_name': 'Dr. Johnson', 'time': '2024-07-01 09:30', 'fee': 120}
{'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'fee': 150}
```

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

Code :

week 12 > railway.py > search_ticket

```python
class Ticket:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date
def search_ticket(tickets, ticket_id):
    """
    Searches for a ticket by its ID.

    Time Complexity: O(n) - where n is the number of tickets.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    tickets (list): A list of Ticket objects.
    ticket_id (str): The ID of the ticket to search for.

    Returns:
    Ticket: The ticket object if found, otherwise None.
    """
    for ticket in tickets:
        if ticket.ticket_id == ticket_id:
            return ticket
    return None
```

week 12 > railway.py > search_ticket

```python
def sort_tickets(tickets, key):
    Sorts tickets based on a specified key (either 'travel_date' or 'seat_number').

    Time Complexity: O(n log n) - where n is the number of tickets.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    tickets (list): A list of Ticket objects.
    key (str): The key to sort by ('travel_date' or 'seat_number').

    Returns:
    list: The sorted list of Ticket objects.
    """
    return sorted(tickets, key=lambda x: getattr(x, key))
```

```python
# Test cases
if __name__ == "__main__":
    tickets = [
        Ticket('T001', 'John Doe', 'Train A', '1A', '2024-07-01'),
        Ticket('T002', 'Jane Doe', 'Train B', '2B', '2024-07-02'),
        Ticket('T003', 'Alice Smith', 'Train A', '1B', '2024-07-01')
    ]

    # Test search_ticket
    print(search_ticket(tickets, 'T002').passenger_name)  # Should return 'Jane Doe'
    print(search_ticket(tickets, 'T004'))  # Should return None

    # Test sort_tickets by travel_date
    sorted_by_date = sort_tickets(tickets, key='travel_date')
    print("Tickets sorted by travel date:")
    for ticket in sorted_by_date:
        print(f"{ticket.passenger_name} - {ticket.travel_date}")

    # Test sort_tickets by seat_number
    sorted_by_seat = sort_tickets(tickets, key='seat_number')
    print("Tickets sorted by seat number:")
    for ticket in sorted_by_seat:
        print(f"{ticket.passenger_name} - {ticket.seat_number}")
```

Output :

```
Jane Doe
None
Tickets sorted by travel date:
John Doe - 2024-07-01
Alice Smith - 2024-07-01
Jane Doe - 2024-07-02
Tickets sorted by seat number:
John Doe - 1A
Alice Smith - 1B
Jane Doe - 2B
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

Code :

week 12 > hostel.py > ...

```python
class Allocation:
    def __init__(self, student_id, room_number, floor, allocation_date):
        self.allocation_date = allocation_date
def search_allocation(allocations, student_id):
    """
    Searches for allocation details by student ID.

    Time Complexity: O(n) - where n is the number of allocations.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    allocations (list): A list of Allocation objects.
    student_id (str): The ID of the student to search for.

    Returns:
    Allocation: The allocation object if found, otherwise None.
    """
    for allocation in allocations:
        if allocation.student_id == student_id:
            return allocation
    return None
```

```python
def sort_allocations(allocations, key):
    """Sorts allocations based on a specified key (either 'room_number' or 'allocation_date').

    Time Complexity: O(n log n) - where n is the number of allocations.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.
```

```python
    Parameters:
    allocations (list): A list of Allocation objects.
    key (str): The key to sort by ('room_number' or 'allocation_date').

    Returns:
    list: The sorted list of Allocation objects.
    """
    return sorted(allocations, key=lambda x: getattr(x, key))
```

```python
# Test cases
if __name__ == "__main__":
    allocations = [
        Allocation('S001', '101', '1st Floor', '2024-07-01'),
        Allocation('S002', '102', '1st Floor', '2024-07-02'),
        Allocation('S003', '201', '2nd Floor', '2024-07-01')
    ]

    # Test search_allocation
    print(search_allocation(allocations, 'S002').room_number)  # Should return '102'
    print(search_allocation(allocations, 'S004'))  # Should return None

    # Test sort_allocations by room_number
    sorted_by_room = sort_allocations(allocations, key='room_number')
    print("Allocations sorted by room number:")
    for allocation in sorted_by_room:
        print(f"{allocation.student_id} - {allocation.room_number}")

    # Test sort_allocations by allocation_date
    sorted_by_date = sort_allocations(allocations, key='allocation_date')
    print("Allocations sorted by allocation date:")
    for allocation in sorted_by_date:
        print(f"{allocation.student_id} - {allocation.allocation_date}")
```

Output :

```
102
None
Allocations sorted by room number:
S001 - 101
S002 - 102
S003 - 201
Allocations sorted by allocation date:
S001 - 2024-07-01
S003 - 2024-07-01
S002 - 2024-07-02
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

Code :

week 12 > movie.py > ...

```python
class Movie:
    def __init__(self, movie_id, title, genre, rating, release_year):
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year
def search_movie(movies, movie_id):
    """
    Searches for a movie by its ID.

    Time Complexity: O(n) - where n is the number of movies.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    movies (list): A list of Movie objects.
    movie_id (str): The ID of the movie to search for.

    Returns:
    Movie: The movie object if found, otherwise None.
    """
    for movie in movies:
        if movie.movie_id == movie_id:
            return movie
    return None
```

```python
def sort_movies(movies, key):
    """
    Sorts movies based on a specified key (either 'rating' or 'release_year').
    Time Complexity: O(n log n) - where n is the number of movies.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    movies (list): A list of Movie objects.
    key (str): The key to sort by ('rating' or 'release_year').
```

```python
    Returns:
    list: The sorted list of Movie objects.
    """
    return sorted(movies, key=lambda x: getattr(x, key))
# Test cases
if __name__ == "__main__":
    movies = [
        Movie('M001', 'Inception', 'Sci-Fi', 8.8, 2010),
        Movie('M002', 'The Godfather', 'Crime', 9.2, 1972),
        Movie('M003', 'The Dark Knight', 'Action', 9.0, 2008)
    ]

    # Test search_movie
    print(search_movie(movies, 'M002').title)  # Should return 'The Godfather'
    print(search_movie(movies, 'M004'))  # Should return None

    # Test sort_movies by rating
    sorted_by_rating = sort_movies(movies, key='rating')
    print("Movies sorted by rating:")
    for movie in sorted_by_rating:
        print(f"{movie.title} - {movie.rating}")

    # Test sort_movies by release_year
    sorted_by_year = sort_movies(movies, key='release_year')
    print("Movies sorted by release year:")
```

Output :

```
e.py
The Godfather
None
Movies sorted by rating:
Inception - 8.8
The Dark Knight - 9.0
The Godfather - 9.2
Movies sorted by release year:
The Godfather - 1972
The Dark Knight - 2008
Inception - 2010
```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

Code :

week 12 > agri.py > search_crop

```python
class Crop:
    def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_estimate):
        self.crop_name = crop_name
        self.soil_moisture = soil_moisture
        self.temperature = temperature
        self.yield_estimate = yield_estimate
def search_crop(crops, crop_id):
    """
    Searches for crop details by crop ID.

    Time Complexity: O(n) - where n is the number of crops.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    crops (list): A list of Crop objects.
    crop_id (str): The ID of the crop to search for.

    Returns:
    Crop: The crop object if found, otherwise None.
    """
    for crop in crops:
        if crop.crop_id == crop_id:
            return crop
    return None
```

```python
def sort_crops(crops, key):
    """Sorts crops based on a specified key (either 'soil_moisture' or 'yield_estimate').

    Time Complexity: O(n log n) - where n is the number of crops.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    crops (list): A list of Crop objects.
    key (str): The key to sort by ('soil_moisture' or 'yield_estimate').
```

```python
    Returns:
    list: The sorted list of Crop objects.
    """
    return sorted(crops, key=lambda x: getattr(x, key))
# Test cases
if __name__ == "__main__":
    crops = [
        Crop('C001', 'Wheat', 30, 25, 1000),
        Crop('C002', 'Corn', 40, 28, 1500),
        Crop('C003', 'Rice', 35, 30, 1200)
    ]

    # Test search_crop
    print(search_crop(crops, 'C002').crop_name)  # Should return 'Corn'
    print(search_crop(crops, 'C004'))  # Should return None

    # Test sort_crops by soil_moisture
    sorted_by_moisture = sort_crops(crops, key='soil_moisture')
    print("Crops sorted by soil moisture:")
    for crop in sorted_by_moisture:
        print(f"{crop.crop_name} - {crop.soil_moisture}")

    # Test sort_crops by yield_estimate
    sorted_by_yield = sort_crops(crops, key='yield_estimate')
    print("Crops sorted by yield estimate:")
```

Output :

```
Corn
None
Crops sorted by soil moisture:
Wheat - 30
Rice - 35
Corn - 40
Crops sorted by yield estimate:
Wheat - 1000
Rice - 1200
Corn - 1500
```

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

Code :

week 12 > flight.py > ...

```python
class Flight:
    def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status
def search_flight(flights, flight_id):
    """
    Searches for flight details by flight ID.

    Time Complexity: O(n) - where n is the number of flights.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.

    Parameters:
    flights (list): A list of Flight objects.
    flight_id (str): The ID of the flight to search for.

    Returns:
    Flight: The flight object if found, otherwise None.
    """
    for flight in flights:
        if flight.flight_id == flight_id:
            return flight
    return None
```

```python
def sort_flights(flights, key):
    """Sorts flights based on a specified key (either 'departure_time' or 'arrival_time').

    Time Complexity: O(n log n) - where n is the number of flights.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.
```

```python
    Parameters:
    flights (list): A list of Flight objects.
    key (str): The key to sort by ('departure_time' or 'arrival_time').

    Returns:
    list: The sorted list of Flight objects.
    """
    return sorted(flights, key=lambda x: getattr(x, key))
# Test cases
if __name__ == "__main__":
    flights = [
        Flight('F001', 'Airline A', '2024-07-01 10:00', '2024-07-01 12:00', 'On Time'),
        Flight('F002', 'Airline B', '2024-07-01 11:00', '2024-07-01 13:00', 'Delayed'),
        Flight('F003', 'Airline C', '2024-07-01 09:00', '2024-07-01 11:00', 'On Time')
    ]

    # Test search_flight
    print(search_flight(flights, 'F002').airline_name)  # Should return 'Airline B'
    print(search_flight(flights, 'F004'))  # Should return None

    # Test sort_flights by departure_time
    sorted_by_departure = sort_flights(flights, key='departure_time')
    print("Flights sorted by departure time:")
    for flight in sorted_by_departure:
        print(f"{flight.flight_id} - {flight.departure_time}")

    # Test sort_flights by arrival_time
    sorted_by_arrival = sort_flights(flights, key='arrival_time')
    print("Flights sorted by arrival time:")
```

Output :

```
Airline B
None
Flights sorted by departure time:
F003 - 2024-07-01 09:00
F001 - 2024-07-01 10:00
F002 - 2024-07-01 11:00
Flights sorted by arrival time:
F003 - 2024-07-01 11:00
F001 - 2024-07-01 12:00
F002 - 2024-07-01 13:00
```